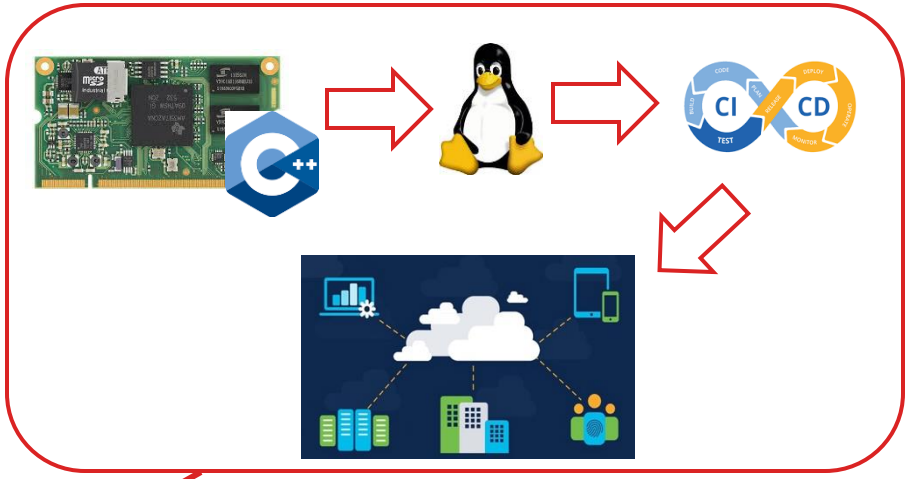
die Mobiliar

Building Crossplane Provider with Go and Bazel

Michel Meyer, 03.09.2026 > https://www.meetup.com/berner-go-meetup/

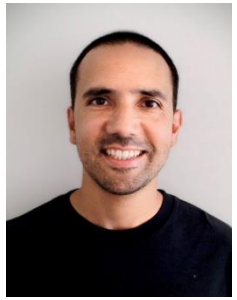
About Me: Michel Meyer



Over 2 years @DieMobiliar in DPT
(Delivery Platform Team)

On Contract

disroop.ch



Me

My Dev-Journey

www.linkedin.com/in/michel-meyer-77177984

www.github.com/mymichu

About Delivery Platform Team

Software Development

Feature Teams

System Teams

Delivery Platform Team

Responsibilities:
- Development Process
- Build-Infrastructure
- Go Stack
- Cloud Landing Zone
- ...

Cloud Landing Zone
- Self Service
- Provisions/Manages
  - Roles / Access
  - Azure Subscriptions
  - Gitlab Structure
  - Kubernetes Clusters

CLZ V1:
- Self-developed
- Micro-Service Architecture

Issues:
- Knowledge not in Mobiliar anymore
- Hard to maintain
- Unreliable

CLZ PoC:
- DPT used Crossplane
  - K8S knowledge in Mobiliar
  - Auto-Reconciliation
  - Given Observability
  - ...

Note: DPT CLZ Solution is **not used**

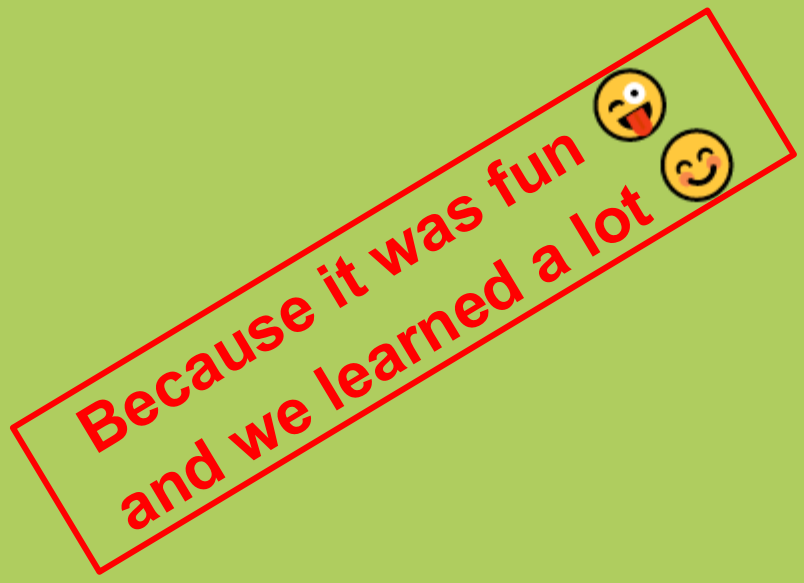
Reason:

The DPT CLZ solution does not align with the die Mobiliar strategy.

SaaS before PaaS before IaaS

SaaS solution is still in evaluation…

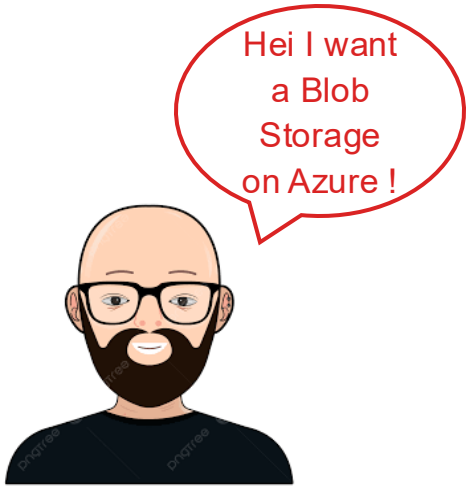
Why are we presenting this topic?



What is Crossplane?

Crossplane is a **framework** for creating CRDS in K8S to provision infrastructure.

What is Crossplane?



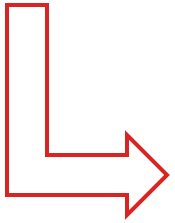
Crossplane Provider

Go

+

Crossplane

=

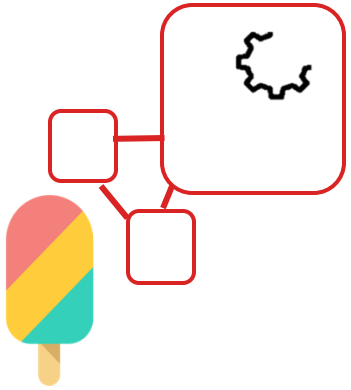


Definition of Infrastructure

+

K8s

+



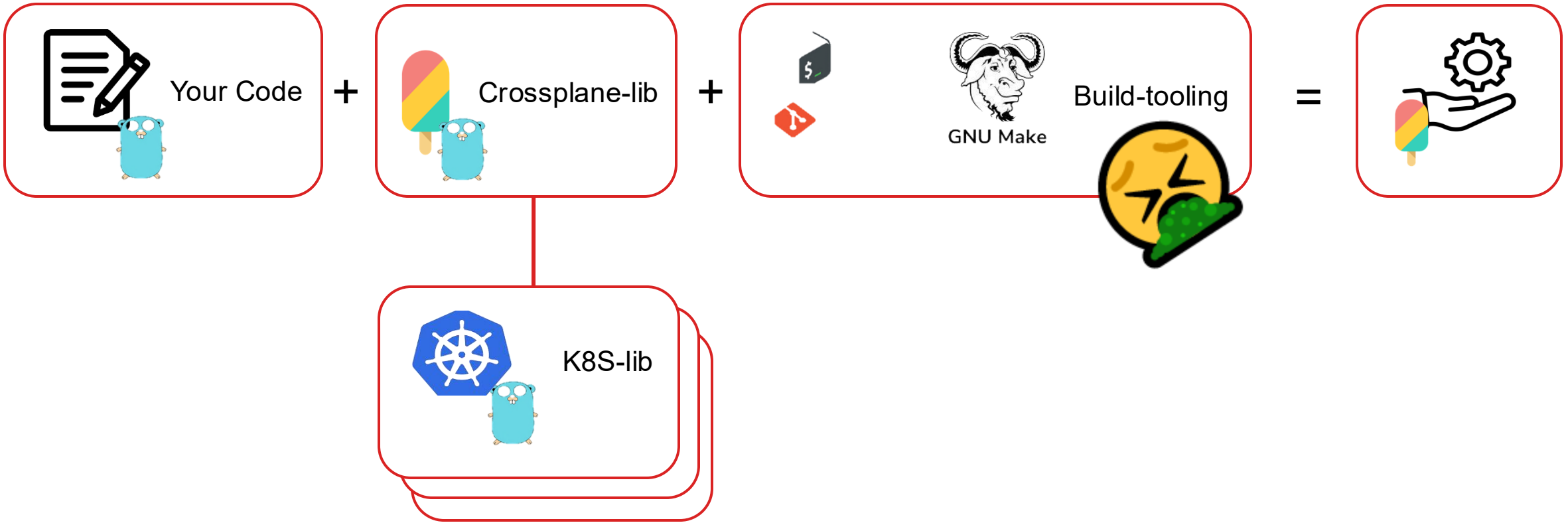
Crossplane

=

Azure Blob Storage

Platform Team Magic Machine

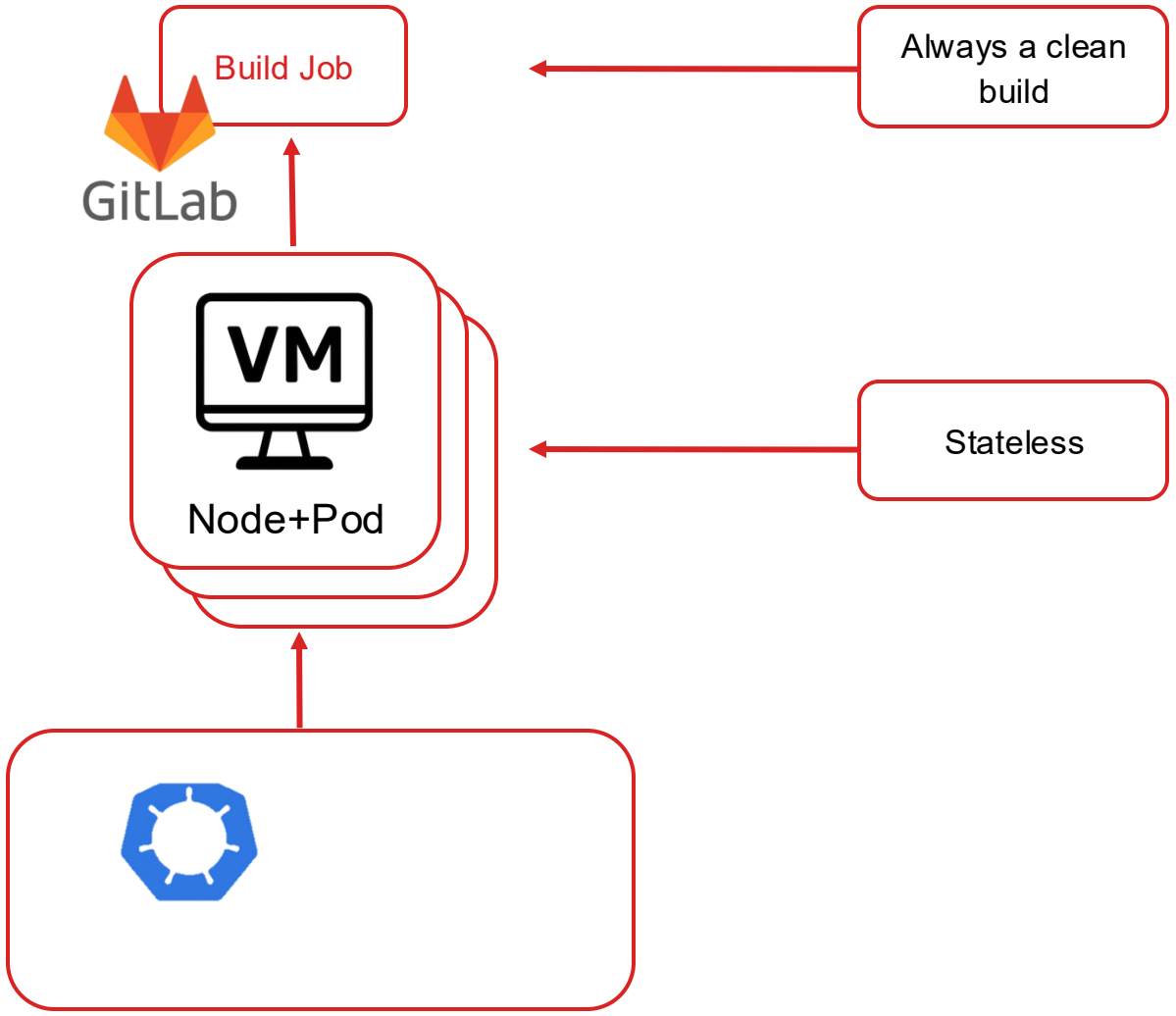
How do we develop Crossplane Providers? The Standard way.



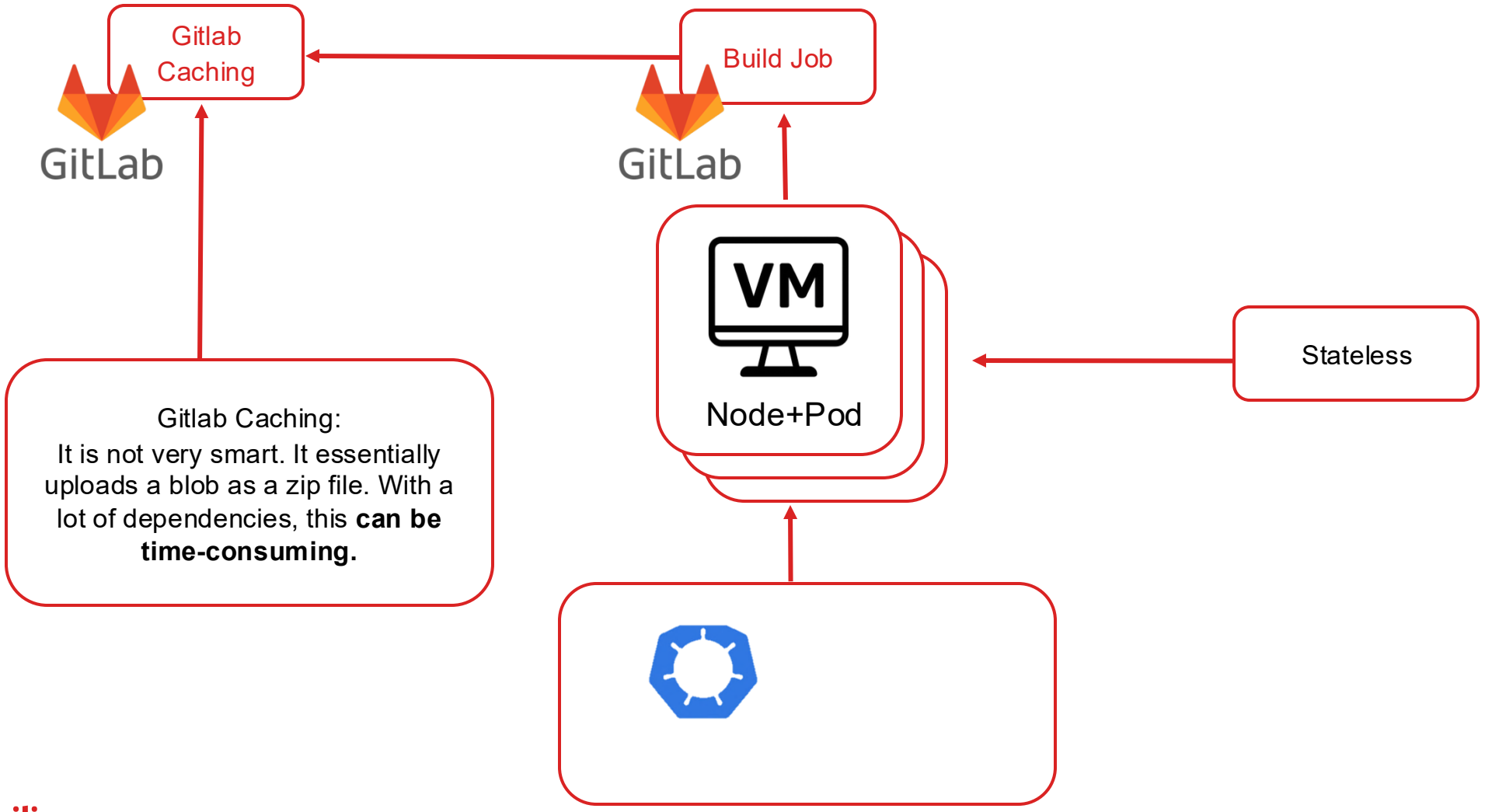
13 Reasons why. I don't like the build tooling of Crossplane

1. Git submodules: Always require an extra step > Error prone
2. Git submodules: CI/CD needs special configuration
3. Makefile: Cryptic syntax > Error prone
4. Makefile: No dependency management > Error prone
5. Makefile: Poor reusability
6. Bash: Cryptic syntax 🤣 > Error prone
7. Bash: Hard to test
8. ..
9. ..
10...
11...
12...
13. It is slow in a CI/CD environment and difficult to integrate caching.

Why is caching important for DPT



Caching with Gitlab Tools



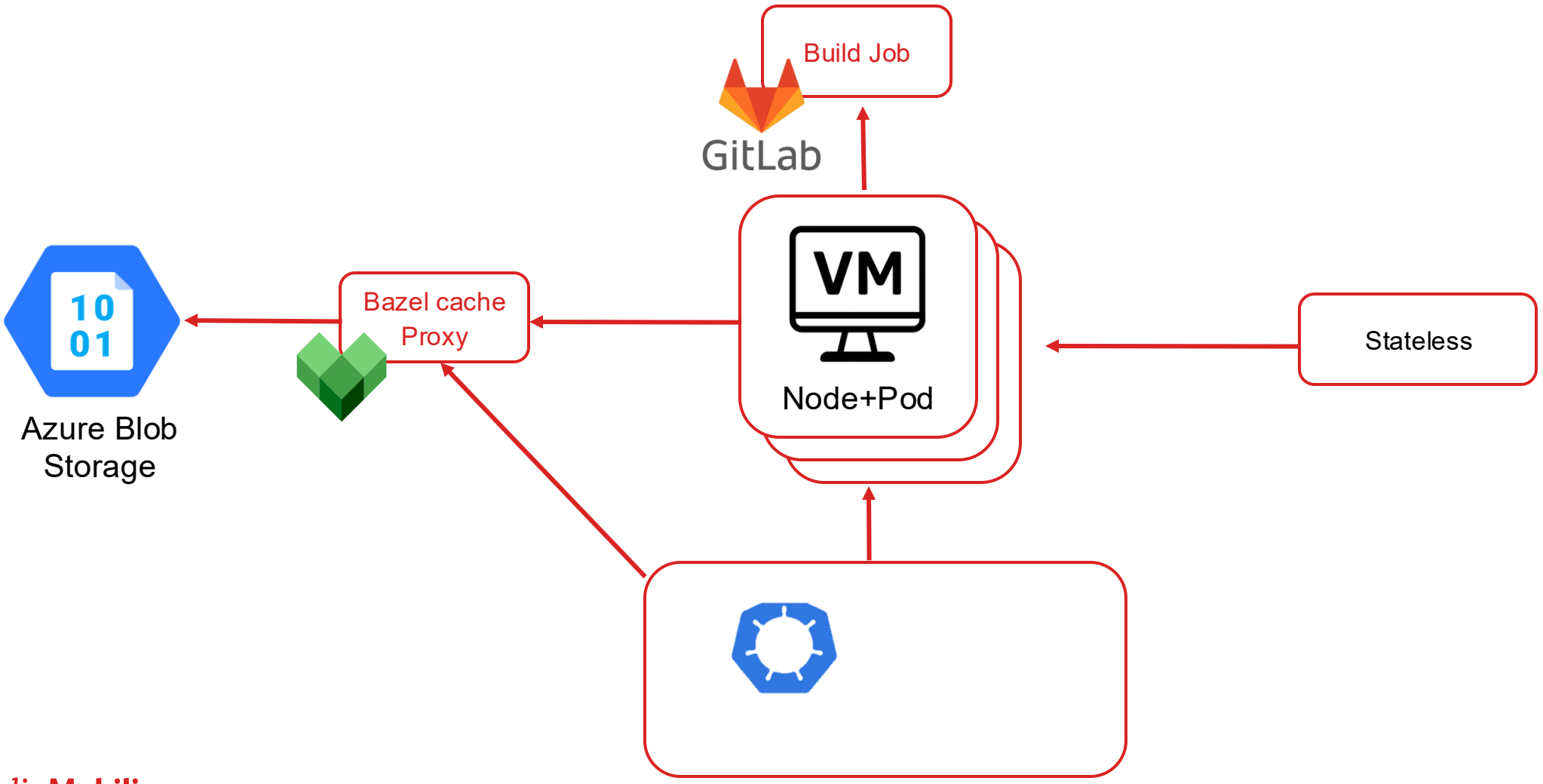
We need a better solution.

Let's do Bazel.

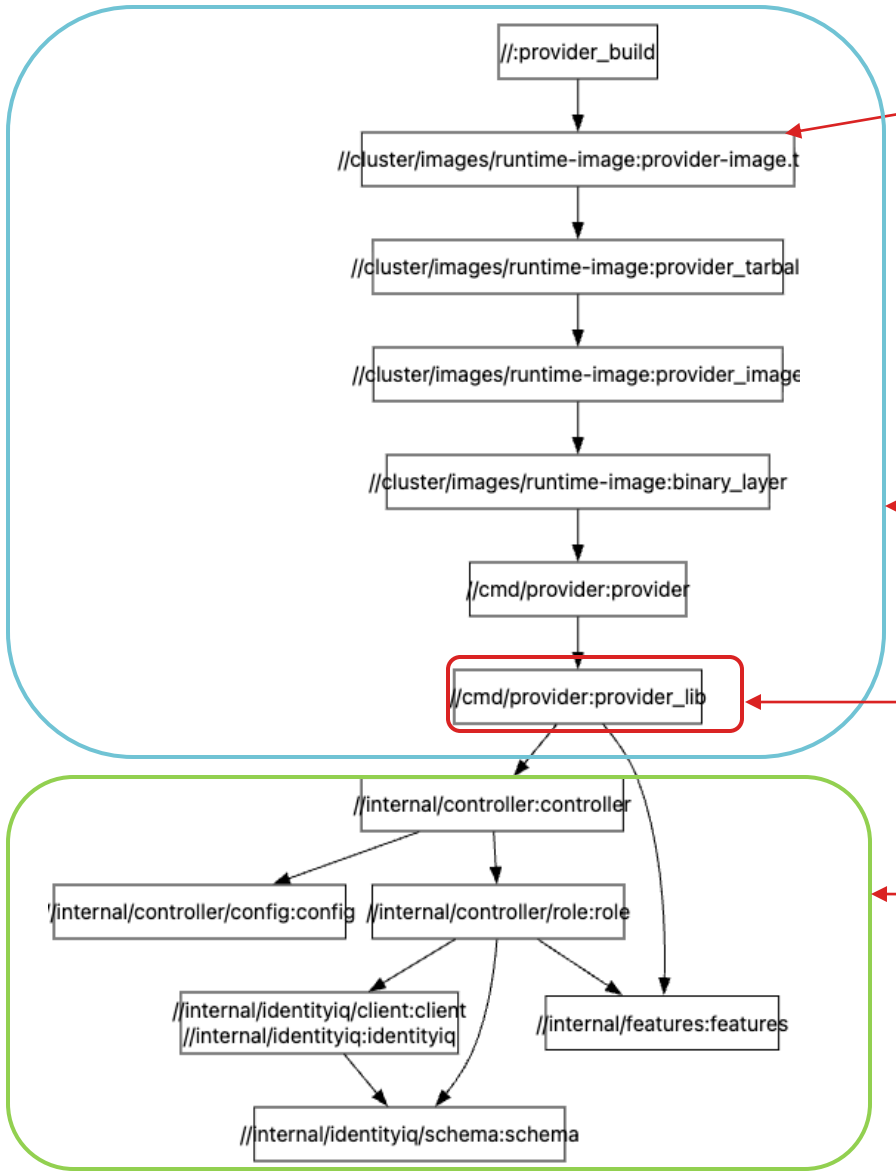
Bazel really? Is it not to complicated?

Hmm… 🧐 I have heard that, but unless we give it a go, we'll never find out.

Let's do it with Bazel



Why is Bazel better? Because it is smarter.



Creates fine-grained packages and a dependency tree.

🔨 Needs a rebuild

⚡ Change

Change the controller lib; all the other libs (internal and external) can be reused.

✅ Can be reused

We get a performance gain. Yay! In our case, we achieved a time gain of around 40%.

Bazel adds complexity: Gazelle and others....

main / clz-provider-identityiq-service / cmd / provider

provider

fix kinpin deps
Michel Meyer authored 6 months ago

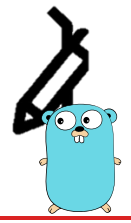
Code owners 1 › Show all

Name	Last commit
..	
BUILD.bazel	fix kinpin deps
main.go	fix kinpin deps

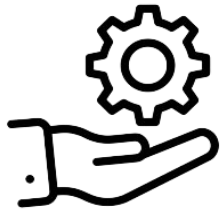
BUILD.bazel 1.33 KiB

```
load("@io_bazel_rules_go//go:def.bzl", "go_binary", "go_library")

go_library(
    name = "provider_lib",
    srcs = ["main.go"],
    importpath = "gitlab.com/diemobiliar/swe/clz/provider/clz-provider-identityiq-service/cmd/provider",
    visibility = ["//visibility:private"],
    deps = [
        "//apis",
        "//apis/v1alpha1",
        "//internal/controller",
        "//internal/features",
        "@com_github_alecthomas_kingpin_v2//:kingpin",
        "@com_github_crossplane_crossplane_runtime//apis/common/v1:common",
        "@com_github_crossplane_crossplane_runtime//pkg/controller",
        "@com_github_crossplane_crossplane_runtime//pkg/feature",
        "@com_github_crossplane_crossplane_runtime//pkg/logging",
        "@com_github_crossplane_crossplane_runtime//pkg/ratelimiter",
        "@com_github_crossplane_crossplane_runtime//pkg/resource",
        "@io_k8s_apimachinery//pkg/api/errors",
        "@io_k8s_apimachinery//pkg/apis/meta/v1:meta",
        "@io_k8s_client_go//tools/leaderelection/resourcelock",
        "@io_k8s_sigs_controller_runtime//:controller-runtime",
        "@io_k8s_sigs_controller_runtime//pkg/cache",
        "@io_k8s_sigs_controller_runtime//pkg/log/zap",
        "@org_uber_go_zap//zapcore",
    ],
)

go_binary(
    name = "provider",
    embed = [":provider_lib"],
    visibility = ["//visibility:public"],
)
```

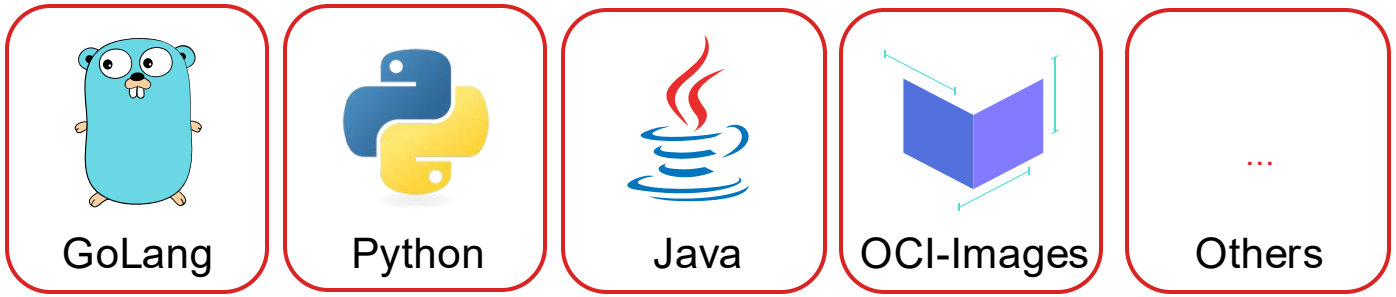
starlark

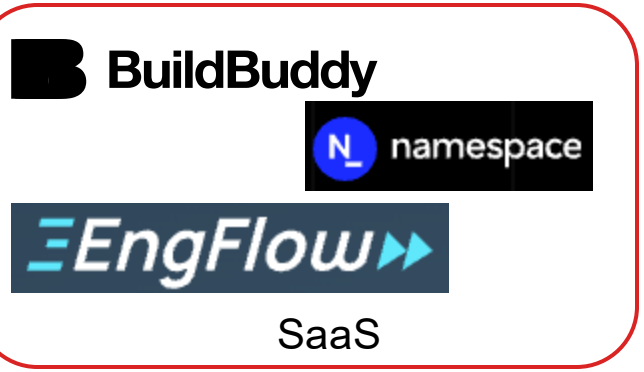


\+ bazel-gazelle + Bazel =



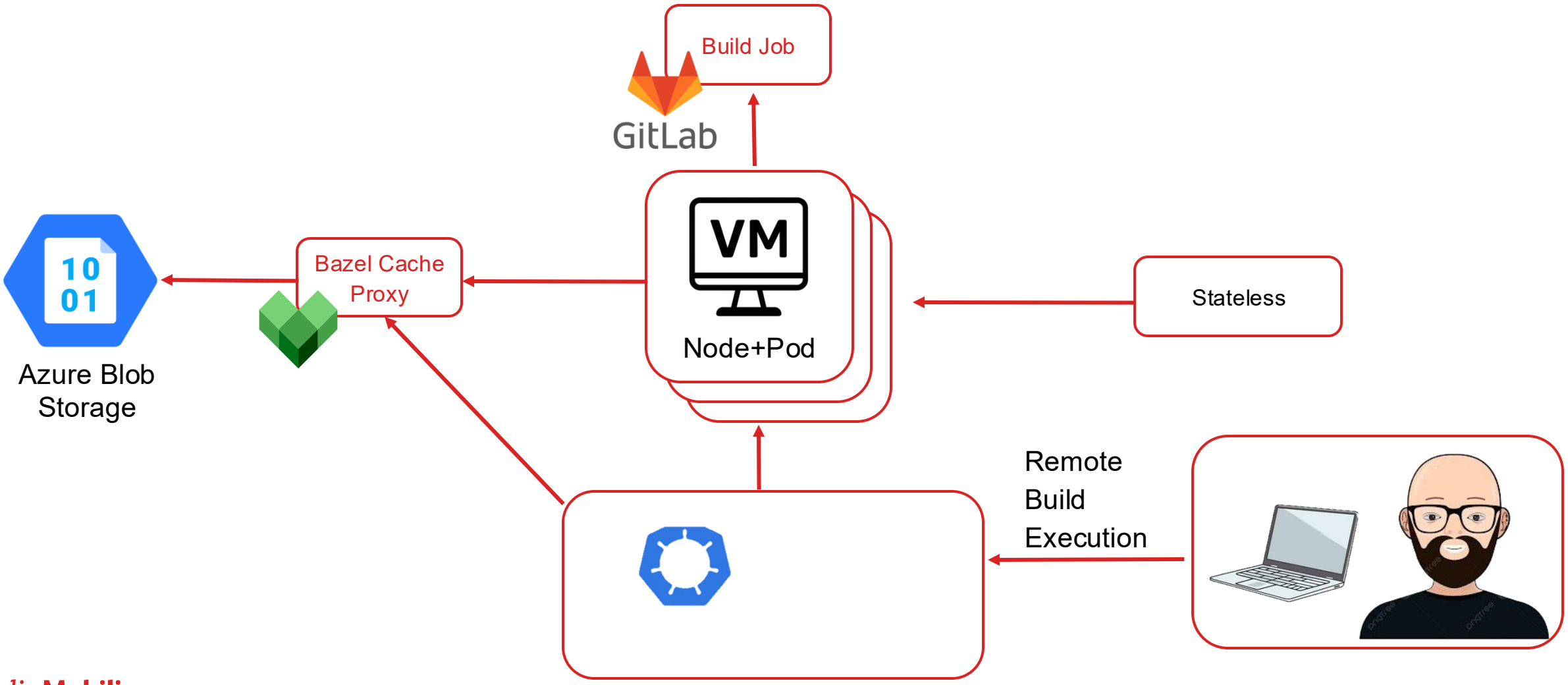
Why does Bazel open up other possibilities? Standardisation leads to reuse





https://registry.bazel.build/

Why does Bazel open up other possibilities? Prod/Dev Gap gets smaller



Yes, it is complicated and the learning curve is steep, but the advantages of Bazel outweigh the disadvantages.

Thank you